Compiler Front-End: Remaining Work and Completion Plan

Status date: 2026-11-03

Scope: Indicode compiler front end only. This plan replaces the previous all-area blueprint PDF.

For the consolidated progress, completed-work summary, percentage breakdown, and remaining roadmap, see the front-end status report and its PDF.

What “100%” means here

The front end reads source, checks it, reports source-aware errors, and produces a semantically valid, independently verified Indicode HIR. It does not generate LLVM IR, machine code, executables, or run programs.

The current progress report calculates a 97.10% weighted estimate, displayed as 97%. The estimated gap is 2.90 percentage points on that judgment-based scale. This is not a claim that 97% of the language specification or 97% of the PDF catalogue is implemented.

The old PDF described a 777-concept catalogue across the whole language/compiler project. Only 40 catalogue entries (IDs 445–484) are in its compiler-front-end section. In the current implementation audit:

Front-end catalogue status	Count
Implemented to the v1 scoped acceptance level	30
Partially implemented; more acceptance work remains	0
Not implemented yet	10
Front-end catalogue total	40
Still unfinished (partial + not implemented)	10

These inventory counts and the 97.10% weighted estimate measure different things. The remaining 737 entries in the full catalogue are outside this front-end-only plan; this is not a claim that they are complete.

Remaining catalogue entries

“Done” means the concept has an implementation and focused test for the documented v1 scope. “Partial” means a subset exists, but the listed completion evidence is still required. “Not started” means there is no implementation to accept yet. Features excluded from v1 remain visible in their own catalogue rows rather than being silently counted as implemented.

ID	PDF concept	Audit status	Work / evidence still needed
445	Source manager	Done	Source ownership, source IDs, lookup, and bounds tests are implemented.
446	Source buffer	Done	Multiple owned source buffers and stable const access are tested.
447	Source span	Done	Source locations/spans are represented and location boundaries are tested.
448	Line table	Done	CR, LF, and CRLF line boundaries, including trailing empty lines, are tested.
449	Column mapping	Done	Byte offsets, UTF-8 scalar columns, and tab stops are distinct and tested; terminal width policy is tracked under diagnostics.

ID	PDF concept	Audit status	Work / evidence still needed
450	UTF-8 validator	Done	Malformed UTF-8 is diagnosed in source tokens, strings, and comments.
451	Token kind	Done	Public token kinds cover every v1 punctuation/operator and invalid/EOF states; keywords have a typed 14-value inventory, and comments are represented as trivia.
452	Token payload	Done	Keywords, radix integers, normalized floats, and escape-preserving string contents have typed payloads; focused tests also verify original spelling and source spans.
453	Lexer state	Done	EOF handling is deterministic; malformed-input recovery and lexer robustness are covered by focused and smoke tests.
454	Trivia preservation	Done	Whitespace and line/nested-block comments retain exact source spelling and spans separately from parser tokens; block-comment newline separators remain tokens.
455	Parser state	Done	Parser position and error state are explicit; boundary, malformed-input, and progress tests pass.
456	Recursive descent parser	Done	Implements every declaration, statement, and type form accepted by the normative v1 grammar; imports and generic declarations are excluded. Recovery is completed and tracked separately under 464–465.
457	Pratt expression parser	Done	Normative precedence/associativity and postfix rules have coverage for every binary operator, prefix nesting, and member/index binding.
458	Grammar version	Done	Public grammar version 1.0.0 matches the normative syntax specification; syntax contract and compatibility policy are documented and checked.
459	AST context	Done	AST_CONTEXT_V1.md specifies that **Program** owns the AST, parser tokens are borrowed only during parsing, and source text must be retained separately to render spans. A returned program is tested after parser/source destruction.
460	AST node arena	Not started	Use an arena or document/measure an alternative ownership design; test destruction and large-input behavior.
461	AST visitor	Done	Public **AstVisitor** has required callbacks for every declaration, statement kind, and expression kind; **walk_ast** traverses the full tree and tests assert complete kind coverage.
462	AST printer	Done	Deterministic golden output covers every v1 declaration, statement, expression kind, prefix operator, and binary operator.
463	AST serializer	Done	AST JSON schema version 1 serializes declarations, node kinds, names, arguments, and source spans; structure, escaping, and schema-version output are tested. Cross-version compatibility guarantees remain part of broader release policy.
464	Syntax recovery	Done	Declaration, statement, and delimited-expression recovery preserve later declarations and valid sibling expressions; malformed-input tests verify diagnostics and retained AST nodes.
465	Error synchronization	Done	ERROR_RECOVERY_V1.md specifies synchronization boundaries; regression tests cover multiple errors and cascading-error suppression.
466	Symbol identity	Done	Stable per-analysis identities cover v1 values, functions, record types/fields, builtin types/functions, and references, with HIR identity verification. Module/import/overload identities are excluded by v1 and tracked under 468–470.
467	Scope table	Done	SEMANTICS_V1.md specifies v1 value/function/type namespaces, visibility, shadowing, and block/function lifetime; nested shadowing and HIR scope checks are tested. Module scopes are excluded by v1.
468	Module resolver	Not started	Define module identity/search roots/cycles and implement deterministic resolution with positive and negative tests.
469	Import resolver	Not started	Implement import paths, aliases, visibility, ambiguity, and missing-import diagnostics.
470	Overload candidate set	Not started	Define overload selection and ambiguity rules; test exact, failed, and ambiguous candidate resolution.
471	Name binding	Done	Every supported v1 variable, function, record, field, and type reference is resolved to an identity; unknown and duplicate references/declarations have stable diagnostics. Ambiguous/inaccessible references cannot occur without the v1-excluded imports/overloads and are tracked under 468–470.
472	Type checker	Done	SEMANTICS_V1.md defines the supported v1 typing rules; semantic and HIR tests cover declarations, expressions, operators, calls, conversions, collections, records, tensors, and stable negative diagnostics.
473	Constraint solver	Not started	If generics/constraints are in v1, implement constraint collection/solving and inconsistent/unsatisfied-constraint tests.
474	Type substitution engine	Not started	If generics are in v1, implement substitution, occurs/consistency checks as required, and nested-generic tests.
475	Constant evaluator	Done	v1 integer/Boolean constant forms, short-circuiting, signed overflow, division-by-zero, and the 256-node evaluation budget are specified and tested.
476	Control-flow validation	Done	Conditions/counts, return typing, reachable statements, and supported branch/loop behavior are specified and tested; loops are conservatively not assumed to execute.
477	Definite return analysis	Done	Nested conditional paths and unreachable statements are tested; the same conservative loop rule is independently enforced by HIR verification.
478	Effect checker	Not started	Define the v1 effect model (or explicitly defer it by a scope decision) and validate declared/used effects.

ID	PDF concept	Audit status	Work / evidence still needed
479	Ownership checker	Not started	Define ownership, borrowing/lifetimes, and unsafe boundaries (or explicitly defer them); add misuse and valid-flow tests.
480	Tensor shape checker	Done	Static rank, numeric element-type, and literal/symbolic dimension equality are tested. v1 explicitly has no tensor operations, broadcasting, or device model, so those compatibility rules do not apply.
481	Diagnostic engine	Done	Secondary notes, stable severity/code policy, multiple-error recovery, applied-fix-it validation, and deterministic text/JSON golden tests are implemented.
482	JSON diagnostics	Done	DIAGNOSTICS_V1.md publishes schema v1; golden tests cover every field, locations, notes, fixes, escaping, ordering, and the version envelope.
483	Language server index	Not started	Build a source-aware symbol/reference index API and test definitions, references, scopes, and invalidation.
484	Incremental parse support	Not started	Define edit ranges/cache invalidation, implement incremental parsing, and prove results match clean full parses.

Work packages and acceptance gates

Implement these in dependency order. A work package is accepted only when its specification, implementation, focused positive/negative tests, and user-facing documentation agree.

A. Freeze the front-end v1 contract

- Document grammar, tokenization, types, integer/float literal rules, conversions, visibility, modules/imports, generics/overloads, tensor checks, ownership/effects, and error behavior.
- For each feature decide: v1 required, explicitly deferred, or out of scope. Do not silently call an undefined feature “complete”.
- Link each normative rule to at least one valid and one invalid test where applicable.

B. Finish source, lexer, parser, and AST infrastructure

- Complete token payloads/trivia and the documented numeric/string/identifier rules.
- Implement the full accepted grammar and precedence/associativity table.
- Complete parser recovery and error synchronization without crashes or loss of later valid declarations.
- Complete AST ownership, visitors, deterministic printing, serialization tests, and schema/version policy.

C. Finish resolution and semantics

- Implement namespaces, visibility, modules/imports, and stable identities across all symbol kinds.
- Implement the accepted overload/generic/type-constraint rules or record a scope decision that defers them.
- Complete constant evaluation, conversions, control-flow/return, ownership/effect, and tensor static checks that remain in v1.
- Keep error codes stable and verify diagnostics at exact source spans.

D. Finish typed HIR

- Lower every accepted v1 construct while preserving resolved identities, types, source spans, and relevant semantic metadata.
- Independently verify all HIR invariants, including symbols, types, control flow, and calls.

- Test AST-to-HIR coverage and JSON round trips/schema compatibility; reject malformed HIR with explicit diagnostics.

E. Finish tooling and release acceptance

- Complete JSON diagnostic schema, fix-it behavior, symbol index, and incremental parse support if retained in v1 scope.
- Add a requirement-linked conformance suite for every v1 grammar/type/diagnostic rule.
- Add coverage-guided lexer/parser fuzzing, sanitizer jobs, clean offline builds, CI, and installed-package consumer checks.
- Ensure all examples and docs match behavior; run all tests from a fresh Release build.

Definition of front-end completion

Report 100% for the agreed v1 front-end scope only after all of the following are true:

1. The v1 contract is versioned and every required feature in the table has an accepted implementation; any catalogue item outside v1 has an explicit scope decision and remains visibly tracked as future work.
2. Every requirement maps to passing tests, including malformed input, diagnostics, recovery, and typed HIR invariants.
3. Full v1 conformance, fuzz, sanitizer, clean-build, and package-consumer checks pass in CI.
4. No known required v1 item remains partial or unimplemented; documentation and acceptance evidence are current.

The current 19-test suite passing is evidence for its tested cases, not by itself proof of 100% language or production readiness. LLVM, machine-code generation, runtime execution, and AI/device backends remain outside this plan.